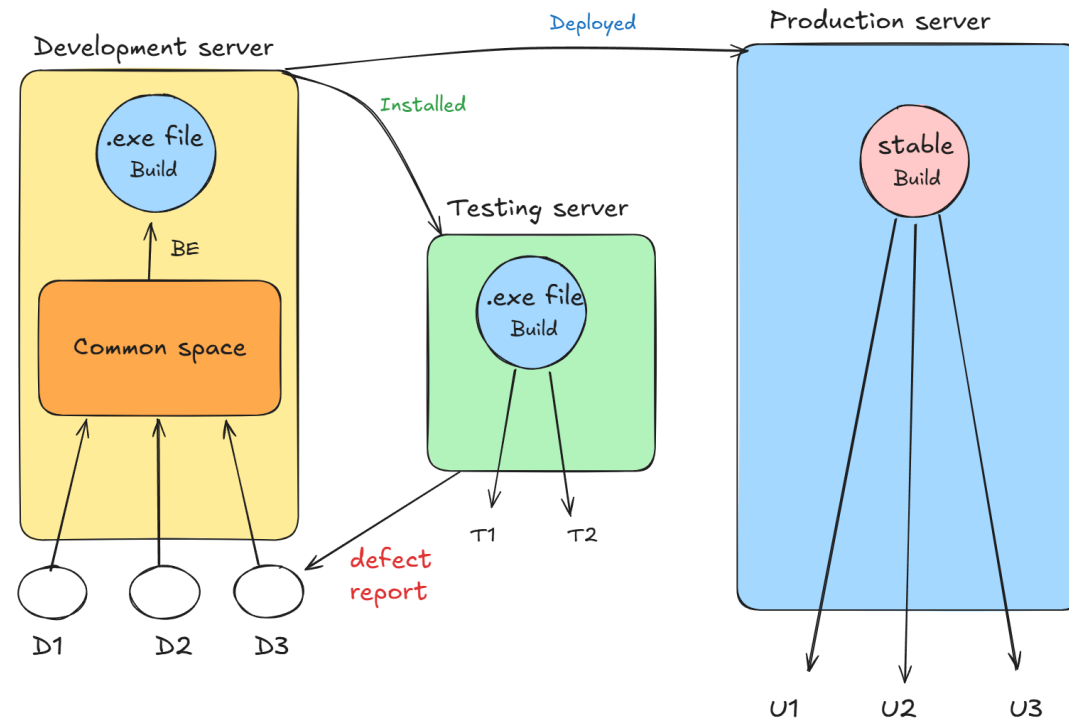


Flow of maven

1. What is traditional approach
2. what is build?
3. what is build tool
4. Alternative build tools
5. Introduction to maven
 - what is maven?
 - History of maven
 - Features of maven
 - Maven repositories
 - Maven life cycle
 - Phases of maven life cycle
6. How to generate the basic .jar and .war file
7. Generate the package in spring petclinic project.
8. Demonstrating sonarqube
9. Demonstrating jacoco
10. Demonstrate nexus.

Traditional approach



1. Collecting the source code from developers
2. Compilation of the code
3. Getting the libraries
4. Run test on source code
5. Convert the source code into an executable file.
6. Testing the application
7. Deploy an application

Drawbacks

1. Lot of manual effort
2. It is time consuming as manually all steps are followed
3. Human errors
4. Inconsistent results
5. Cost increase.
6. Leads to software inefficiency

To overcome the drawbacks we go for build tool.

Build

>> It is the process of converting the source code into an executable file.

-OR-

It is the executable file or a compiled version of a program or an application.

Build tool

>> It is the software which is used to generate the executable file.

-OR-

It is the tool which converts the source code into an executable file.

Process of developing build

>> It is the steps involved in converting the source code into a working software application that includes compiling, linking, packaging & sometimes deploying the software

Why we need build tool ?

- >> It is used to automate the process of converting the code otherwise developers have to manually compile, link, test & package the code everytime they make the changes.
- Faster development
 - Consistency : Ensuring the program is built the same way everytime reducing the errors.
 - Dependency management : Automatically downloads & integrates necessary external libraries.
 - Automation : running the test, check for errors & even deploy the program automatically.
 - Best collaboration

Introduction to maven

Maven

>> It is a build automation tool mainly used for java project.
It helps developers to automate the process of compiling the code, managing the dependencies, running the tests & packaging the application

History of maven

1999-2002 : The struggle with Ant build tool

- Ant was the primary tool but it required the configuration & library handling which became difficult for larger projects.

2002 : Maven was introduced by the founder "Jason van zyl"

- JVZ was a developer at apache created maven to solve all the issues
- maven 1.0(xml based, has limited features) was released in 2002.

2004 : Maven was standardized & it was released globally by apache.

2005 : Maven 2.0 was released

- Introduced pom.xml for better configurations & dependency management.

2009-2010 : Maven 3.0 was released

- Improved performance & stability
- Better integration with IDE like eclipse, netbeans.

Features of maven

1. It is free and open source.
2. Dependency management
 - It downloads the required external libraries(jar files) from online repository from official website.
 - when we install maven it downloads all the basic dependency
 - It handles transitive dependency
3. It has pom.xml
 - Maven uses pom.xml to define project configurations, dependencies & build processes.

POM --> Project object model

XML --> Extensible markup language

- This is an important file in maven which is used to store the information of the project such as pluggins, dependencies or versions of the program.
- without pom.xml , maven cannot work or no maven project will be generated.

HTML	XML
Hyper text markup language	Extensible markup language
HTML contains pre defined tags - <p>, <body>, <html>, 	XML contains user defined tags (It has customized tags) such as <data>, <abc>,
It is used to design webpage & display the content of webpage.	It is used to store & organize the data(not for display).
Usage : Web developement(websites, UI)	Usage : Data exchange(API, configurations)

4. Repositories management
- Maven uses repositories in order to store and retrieve the project dependencies.
 - It is having the repositories like local , central & remote repositories
 - If any dependencies are not found in your local system then maven will go to central/ remote repositories to download it.

5. It is having life cycles such as default, clean & site life cycles.

6. It is easy to use (Maven will complete the entire process automatically that is from compiling to packaging)

7. It supports plugin ecosystem
- It uses plugin to extend the functionalities
Ex : unit testing, code analysis & reporting as well.....

Popular pluggins :

1. Surefire (runs the unit tests)
 2. Compiler (compiles the java code)
 3. Assembly plugin (creates distributable archives like ZIPs & TARs.....)
- Packages the project into custom archive formats (ZIP, TAR & JAR)

What is plugin ?

- >> It is the software or a tool that adds the functionalities to the existing software.
- It extends the maven functionality (compiling, testing & packaging)
 - It is used during the build process.
 - It is executed during maven build phases.
 - Surefire plugin, Shade plugin, Maven compiler plugin.....

8. Build automation

- Maven automates the build lifecycle which includes compiling, testing, packaging & deploying.

* what is POM.XML?

>> It is the core configuration file in maven project. It usually defines the dependencies, build settings & pluggins of project consisting of all the necessary information about the maven project.

* why is POM.XML important?

>> Maven will download all the necessary JAR files instead of doing it manually

- It defines how your project is going to be built & packaged as well.
- It makes the collaboration more easier.

Note : whenever we generate a maven project, POM.XML file is also generated automatically.

* why maven is important ?

>> Maven is widely used, powerful & flexible build tool for java projects.

- It saves time
- It also scales easily
- It integrates with modern tool
- Ensures consistency.

Maven repositories

>> It is the storage place where maven keeps all the dependencies(libraries), pluggins & project build.

There are 3 types of repositories

1. Local repository
2. Central repository
3. Remote repository

1. Local repository

- >> This repository is available locally when we download the maven from the official website
This repository is denoted as .m2 directory in local system.
Once maven downloads the dependency from the central repository, it saves the copy in your local system and that saved folder is called as local repository.
- The local repository (.m2), it will store all the dependencies, pluggins which are available during the download of maven as well as it keeps the build which got generated.
 - By default, it's stored in your system at ;
c:\Users\username\.m2\repository --> Windows
/home/username/.m2/repository --> linux/mac
 - Next time you run the project maven will reuse the already downloaded dependency from the local repository instead of downloading again --> it saves time.
 - It also stores pluggins as well as build output generated by maven.

2. Central repository

- >> This repository is like the official store(global library) of maven where all the pluggins, dependencies are globally available.
This is maintained by the maven community(official website).
In this central repository if we require any additional libraries or pluggins we can easily download it & use it in the projects.

* What does central repository contain?

>> It contains pre-built dependencies(JUnit, spring.....)

3. Remote repository

- >>It is the custom repository where we store the build onto the server.
* In order to store the build in the server we have the tool called as "nexus".

Maven life cycles

>> It is the set of phases that maven uses/runs one by one in order to build and manage the project.

There are 3 main life cycles

1. Default life cycle
2. Clean life cycle
3. Site life cycle

1. Default life cycle

>> This life is used mainly for project development and deployment.

Phases of default life cycle

1. Validate

>> It will validate the project information (name, version, buildID of project & name of the company)

Command : `mvn validate`

2. Compile

>> It will check the syntax of the source code

Command : `mvn compile`

3. Test

>> It will test the source code by running the unit test. It is also called as white box testing and it is done by the developers.

Unit testing --> testing the code line by line.

Command : `mvn test`

4. Packaging

>> Here, the source code is converted into an executable file(.jar, .war, .ear)

Command : `mvn package`

5. Verify

>> It will verify whether the build that is generated is working properly or not before installing or deploying.

Command : `mvn verify`

6. Install

>> Here, it installs the package(build) in your local system(`~/.m2/repository`)

Command : `mvn install`

7. Deploy

>> In this phase, it will deploy the final build into remote repository(nexus).

Command : `mvn deploy`

2. Clean life cycle

>> This life cycle will remove the old build files. It basically handles project cleaning.

Command : `mvn clean`

why it is important ?

--> It helps to prevent the new files and classes from mixing with the old ones to avoid the conflict.

It will make sure the information of the build(test reports, outputs) are accurate.

It helps in keeping the project folder clean.

3. Site life cycle

>> This life cycle in maven is about the project documentation.

It helps in generating the information like :

1. Code coverage report

2. Details of your project(Name, version, dependencies....)

3. Unit test reports

4. If any extra configuration is done then those report will be generate as well.

Command : `mvn site`

Why it is important ?

--> It helps the developers to easily share the project with their team members.

Team will be able to get the reports(code coverage, test report, plugin report, dependencies....)

It helps in project transparency.

EXECUTION

>> Demonstrate a basic jar file generation

1. Create folder & open vc code inside the folder
2. Open new terminal in vs code
3. Generate the maven template by giving the command in terminal.

Command : `mvn archetype:generate`

4. In all the template make sure the default template is selected.
5. Press enter and go for the recent version
6. Enter all the details --> groupID, artifactID, version, package

groupID : name of the company

artifactID : name of the project

7. Confirm the configuration by typing y & make sure that the folder is generated with project name and check whether pom.xml is present or not.
Check whether the source code & test cases are generated by default or not.
8. Get in the folder where pom.xml is present and execute all maven basic commands.
 - > Observe the changes in folder structure
 - > Generate the build, install the build, generate documentation of project & clean the build when needed.

>> Demonstrate a basic war file generation

1. create a folder & open vs code
2. Generate maven templates by giving the command "mvn archetype:generate"
3. Generate maven webapp templates from the generated templates

Command : `maven-archetype-webapp`

4. Choose the required template and the recent version of template & setup the project environment
5. Go to the folder where pom.xml is present and generate the .war package, install, generate the documentation of project & clean whenever required.

Generate a package from the repository(spring petclinic) and check

the output of jar file in localhost

1. Search spring petclinic in the browser.
2. Choose sample repository of spring petclinic.
3. Create folder, open vs code & clone spring petclinic repository.

Command : `git clone <spring petclinic url>`

4. Get in the cloned folder and generate the package .

Command : `mvn package`

5. Expose the generated jar file to the localhost(server).

Command : `java -jar <target_folder>/<build_name>.jar --server.port=<port_number>`

Example : `java -jar ./target/spring-petclinic-3.5.0-SNAPSHOT.jar --server.port=140`

6. Open the browser & check via localhost with port number

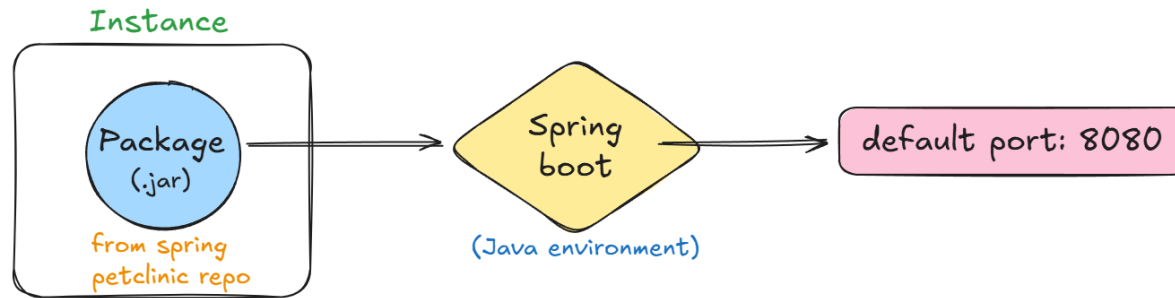
URL : `localhost:<port_number>`

Example : `localhost:140`

Note : Localhost is the systems default IP

>> Can be written as 127.0.0.1 in the form of IP.

Generate the jar file & through AWS check the output of that jar file
by using spring petclinic application



1. Go to AWS and launch ubuntu instance while selecting "t2.medium" instance type.
2. In network settings allow port http as well as ssh
3. Create the keypair, select it and launch the instance.
4. Connect instance and install the software such as openjdk-17-jdk, maven after updating it.

Commands to update and install

```
sudo apt update  
sudo apt install openjdk-17-jdk -y  
sudo apt install maven -y
```

5. Once installation is done, clone .jar file(spring petclinic) from github repository to instance.
6. Expose the port number 8080 in security group section in instance.
7. Get in the cloned folder and generate the package.
8. In instance get in the target folder and expose .jar file through spring boot default port number 8080.

Commands

```
java -jar ./<build_name>.jar
```

9. Copy the public IP of the instance & paste it the browser through port number 8080 to expose the application spring petclinic.

URL

```
http://<public-IP>:8080
```

Execution of sonarqube

1) Launching and Preparing Instances

Create two EC2 instances — one for Maven and another for SonarQube. Use t2.medium as the instance type and select Ubuntu as the operating system. Enable HTTP traffic and generate a key pair while launching the instances & open port 9000 in the security group settings to allow SonarQube access. Connect to the SonarQube instance & prepare a script that installs all the required packages — SonarQube, unzip utility, and Java.

Sample Script Commands:

```
wget -O app.zip <sonarqube-link(download)>
sudo apt install unzip
unzip app.zip
sudo apt update
sudo apt install openjdk-17-jdk -y
```

2) Running SonarQube

Once the files are extracted, navigate to the following directory to locate the sonar.sh script :

```
/home/ubuntu/sonarqube-25.7.0.110598/bin/linux-x86-64
```

Start SonarQube by executing the script:

```
./sonar.sh start
```

Copy the instance public IP and open it in your browser using port 9000:

```
http://<public-IP>:9000
```

Note : Login with the default credentials — username: admin, password: admin. Then update the password after first login.

3) Maven Setup

On the Maven instance, install Maven and Java, clone the Spring PetClinic framework project & build a package using Maven for the PetClinic application.

4) Creating a Project in SonarQube

In SonarQube, click on "Create a project" and choose local project & Provide a project name then a project key will be generated automatically. Confirm to create the project — it will be added to the dashboard then choose local analysis to scan the project on your system & Generate a token for the project and give it a suitable name. Select Maven as the build tool & copy the generated maven command and run it inside the Maven instance. Once the build completes successfully, go back to the SonarQube web UI to view the generated analysis report.

Execution of nexus

1) Launching Instances

Start two EC2 instances — one dedicated to Maven and the other for Nexus, using Ubuntu OS and selecting t2.medium instance type.
Open the required ports — enable HTTP and 8081 for Nexus in the security group & launch both instances and establish an connection to the Nexus server.

2) Installing Nexus

Prepare a shell script that downloads the Nexus archive using wget, extracts it, and sets up the required environment.

Commands:

```
wget -O app.tar <nexus download link>  
tar -xvf app.tar
```

Once the package is extracted, move into the folder and navigate to the binary directory:
/home/ubuntu/nexus-3.32.0-03/bin

Start Nexus by executing the startup script

Take the public IP of the Nexus instance and open it in your browser with port 8081 :
http://<public-IP>:8081

3) Nexus Initial Setup

Log in using the "admin" account at the top of the dashboard & locate the admin password file path, copy the password, and log in using it.
sudo cat <path-to-password-file>

Change the default password to a new one for security, disable anonymous access and confirm changes & complete the initial configuration to access the Nexus dashboard.

4) Maven Setup

On the Maven server, install Maven and openjdk-17-jdk, clone the Spring PetClinic project from GitHub & go inside the cloned directory and build the package using Maven.

5) Configuring Nexus Repository (Web UI)

From the Nexus dashboard, go to Settings → Repositories → Create Repository then select maven2(hosted).
Give your repository a unique name, select mixed then under "Deployment Policy", select allow redeploy & save the repository.

6) Uploading artifacts from Maven

Edit the pom.xml and settings.xml files in the Maven project, add the distribution management section in pom.xml pointing to the Nexus repo URL.
In settings.xml, configure the server credentials (repository ID, username, password), execute the Maven deploy command : mvn deploy
Verify deployment by checking inside the Nexus repository under Browse → confirm that your artifact has been uploaded.

Execution of tomcat

1) Launching and Preparing Instances

Create EC2 instance — Use t2.medium as the instance type and select Ubuntu as the operating system.

Enable HTTP traffic and generate a key pair while launching the instances & open port 8080 in the security group settings to allow tomcat access.

Connect to the instance & prepare a script that installs all the required packages — tomcat, java & maven.

Sample Script Commands:

```
sudo apt update
```

```
sudo apt install openjdk-17-jdk -y
```

```
sudo apt install maven -y
```

```
wget -O app.zip <tomcat-link(download)>
```

Run the script and get all tools installed then extract tomcat folder

Get in the folder and start tomcat then copy the public ip and paste it in new tab to verify if its successfully installed or not.

2) Maven Setup

On the Maven server, install Maven and openjdk-17-jdk, clone the Spring PetClinic project from GitHub & go inside the cloned directory and build the package using Maven.

Get in the cloned project then in target.

Use the command to copy petclinic.war to tomcat webapps :

```
sudo cp ./petclinic.war /(absolute path of webapps)
```

copy the public ip of tomcat and paste it in browser : `http://<public-ip>:8080/petclinic`

Petclinic page should be displayed.

Execution of jacoco

1) Launching and Preparing Instances

Create EC2 instance — Use t2.medium as the instance type and select Ubuntu as the operating system.

Enable HTTP traffic and generate a key pair while launching the instance.

Connect to the instance & prepare a script that installs all the required packages — java & maven.

Sample Script Commands:

```
sudo apt update
```

```
sudo apt install openjdk-17-jdk -y
```

```
sudo apt install maven -y
```

Run the script and get all tools installed.

Note : You can install tools one by one without creating the script as well. You can follow any one method that you feel is simple.

2) Maven Setup

On the Maven server, install Maven and openjdk-17-jdk, clone the Spring PetClinic project from GitHub & go inside the cloned directory and build the package using Maven.

Get in the cloned project then modify the pom.xml file.

Add jacoco plugin in pom.xml in pluggins section below last </pluggin> then save & exit.

Use the command : mvn verify

After verifying get in the target folder then get in the site folder and then jacoco folder --> after using the command "ls" the index.html file will be visible.

3) Installation of nginx

Get in the ubuntu directory by using the command "cd"

Install nginx :

```
sudo apt install nginx
```

Start nginx & check the status of nginx :

```
sudo systemctl start nginx
```

```
sudo systemctl status nginx
```

Get in the cloned folder --> target --> site then copy entire jacoco folder in nginx executable path.

```
sudo cp -r ./jacoco/* /var/www/html
```

Copy the public IP and paste it in the browser

http://<public-ip>:80 ==> 80 is port number of nginx

The coverage report of project generated by jacoco will be displayed.